MODULE 11 · CLASS SLIDES

Vibe Coding for Product Managers

Varun Kumar · SkillNavora

Prepared for Skillnavora · Learn. Practise. Implement. Grow.

Product Management Roadmap

PHASE 1

CRAFT

Modules 1–4

1. Fundamentals of Product Management
2. Product Strategy & Strategic Thinking
3. Problem Discovery, Market & User Research
4. Problem Framing & Prioritization

PHASE 2

CRAFT + AI

Modules 5–8

5. UX & Design Collaboration
6. Effective Communication as a PM
7. Analytics & Metrics
8. Product Launch, Growth & Monetization

PHASE 3

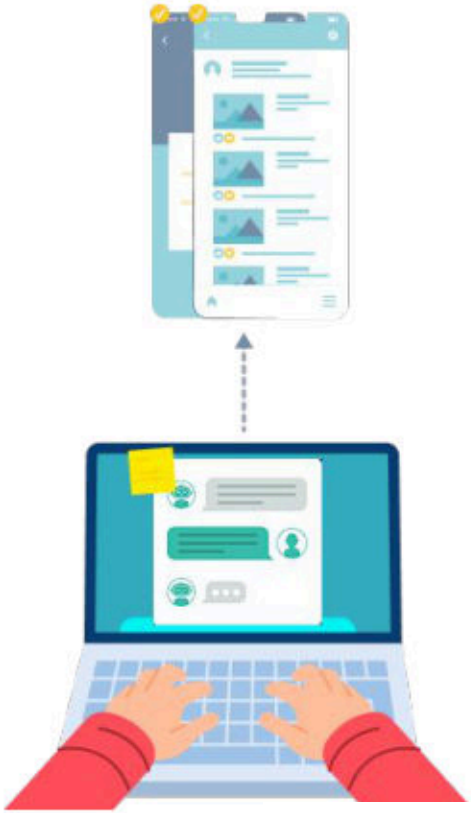
DEEP AI

Modules 9–13

9. Tech 101 & System Design
10. LLMs, RAGs & AI Foundations
11. Vibe Coding for PMs — WEEK 11
12. Agentic AI & AI Systems in Production
13. Capstone Project & Interview Prep

Vibe Coding — Building Without Knowing Code

“How PMs can turn ideas into working tools — through prompts.”



Day 1 :- Agenda

- What problem vibe coding solves for PMs
- What vibe coding is
- How it differs from traditional coding
- Workflow: Prompt → Test → Refine
- Claude Setup and Workflow
- Live demo: your first mini-app

Why Traditional Product Building Slows PMs Down

- PMs wait weeks for design/engineering bandwidth
- Can’t validate hypotheses fast
- Product discovery becomes slow and costly
- Visualizing ideas early is hard



Why PMs Need Vibe Coding

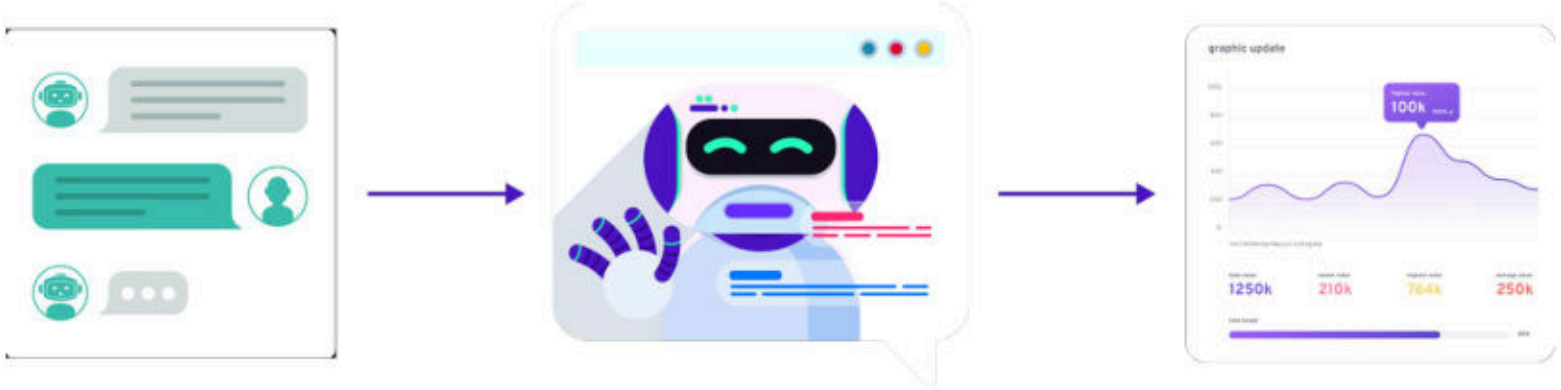
- Build faster prototypes without waiting for tech help
- Communicate clearly with engineers & designers
- Test product logic before sprint planning
- Automate recurring PM tasks (tracking, reporting, prioritization)



What Is Vibe Coding?

- Vibe coding = building apps through conversation with AI.

You describe what you want, refine, and test interactively.



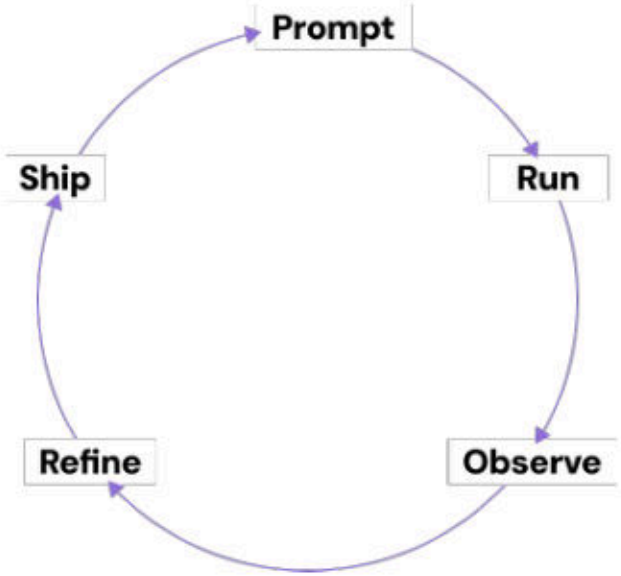
Vibe Coding vs Traditional Coding

Factor	Traditional	Vibe Coding
Language	Syntax-based	Natural language
Workflow	Write → Compile → Debug	Describe → Test → Refine
Users	Engineers	Builders / PMs
Goal	Precision	Speed + Exploration

Vibe Coding Workflow

Key Message:

- It’s iterative — you’ll fail fast, learn fast.
- Don’t aim for perfection — aim for progress.



First prompt is the MOST important step

It's like setting the angle for your rocket 🚀

Difference by even a degree will lead to major displacement from expectations later!

Think of it like a **PRD** you provide to the engg team (or PRP, as I like to call it 😇)

Best way to write the first prompt:

1. 1 line **overview** of what you are building
2. Details about **core MVP** functionality (the primary use case)
   - also include login, auth, DB etc at this stage itself
3. Style details (and preferably **screenshot(s)** of what it should look like)

PRO tip: Refine this prompt using ChatGPT/Claude first before taking it to Lovable/Cursor!

```
Cheat prompt: "Rewrite this prompt to be more concise and detailed:"
```

First prompt is the MOST important step

Basic first prompt template:

I need a [type of application e.g. Shopify app] with [list features]. This will be used by [target audience] to solve [core value proposition]. The app will be successful if it can help the user [achieve X].

Start with the main page containing: [Detailed page requirements].

Use a modern, clean and professional responsive design using React, Tailwind, and Lucide React icons. Add subtle micro-animations and smooth interactions to create a delightful user experience. Provide clear success/error states, loading indicators, and confirmation messages.

And if the first prompt's result is not even 60-70% of what you had in mind -

Do yourself a favor, and **start over.**

With a better PRD and screenshots.

Why not begin with a heavily detailed PRD?

Simply coz AI can't do too many things at once (and tends to create bugs if you try that anyway)

Rule of thumb: Give it **MAX 3 things** to do in 1 prompt

Get in the habit of creating MVPs, and iterating upon each version

When in trouble/doubt - simply **revert** to previous version 😅

Read more about Lovable best practices here: https://lovable.dev/blog/2025-01-16-lovable-prompting-handbook

And where to get screenshots?

Any website/tool/app/product you find similar to what you want to build

- Chatbots? Think ChatGPT, claude, (even lovable itself!)
- Social media? Shamelessly throw in Linkedin, insta, etc
- Forums? Think Reddit, Quora, etc

Other resources (for “inspiration”):

1. https://dribbble.com/
2. https://21st.dev/community/components
3. https://www.behance.net/search/projects

Read more about Lovable best practices here: https://lovable.dev/blog/2025-01-16-lovable-prompting-handbook

Claude Model Lineup

Available models from Haiku to Fable.

Choose the smallest model that reliably does the job.

Haiku

Fastest + cheapest

Use for:

Quick classification, extraction, summarisation, rewrites, first-pass drafts



Sonnet

Balanced default

Use for:

General product work, planning, coding, reviews, debugging



Opus

Deep reasoning

Use for:

Complex problem-solving, architecture, high-stakes analysis



Fable

Long-form synthesis

Use for:

Narratives, rich synthesis, multi-step content generation

When to Use Which Model

Route tasks by risk, complexity, and output quality needs.



Haiku

Best for speed and cost

Use when:

- summarising
- classifying
- extracting fields
- rewriting
- quick first-pass drafts
- low-risk repeatable work



Sonnet

Best default builder model

Use when:

- coding
- planning
- reviews
- debugging
- medium-complexity work
- most day-to-day product and engineering tasks



Opus

Best for deep reasoning

Use when:

- ambiguous problems
- architectural decisions
- critical analysis
- trade-off thinking
- hard bugs
- high-stakes tasks



Fable

Best for long-form synthesis

Use when:

- multi-step narratives
- rich content generation
- synthesising many sources
- shaping polished long-form outputs

ROUTING SUMMARY



Low risk + repeatable → **Haiku**



Default build work → **Sonnet**



Ambiguous or critical → **Opus**



Long-form synthesis → **Fable**

Claude Code Works in Terminal and IDE

Choose the interface that matches how you build.

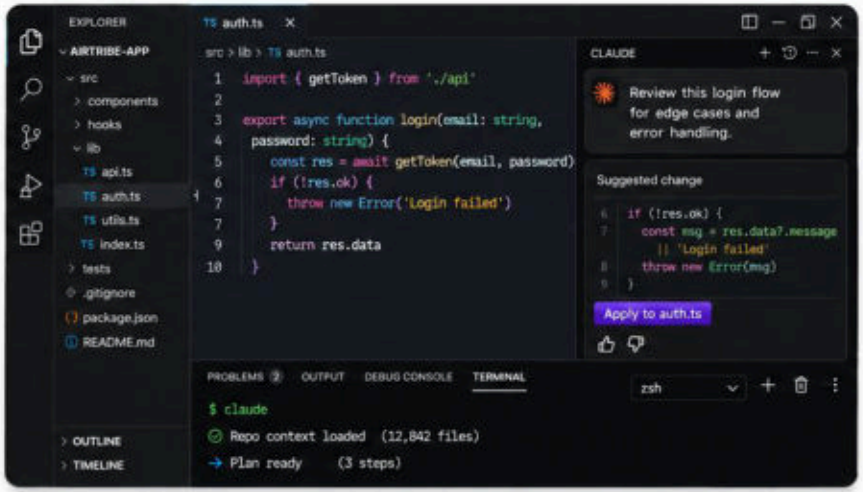
Terminal



Best for quick commands,
repo navigation, small fixes.

Pick based on
workflow.

IDE / VS Code



Best for reviewing changes, editing files,
debugging, and tests.

CLAUDE CODE WORKS INSIDE YOUR REPO

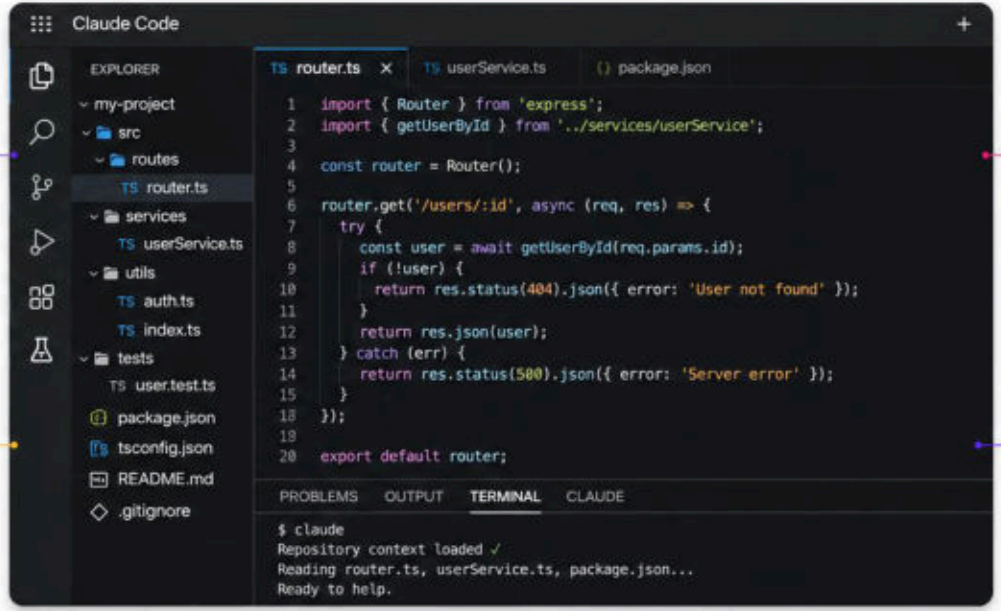
It reads structure, understands files, and helps you make changes with context.

Open files

Sees what you're working on.

Dependencies

Knows your packages, scripts, and imports.



Tests

Understands your test coverage and intent.

Suggested changes

Proposes edits with context, not guesses.

Understands codebase

Reads related files

Plans changes

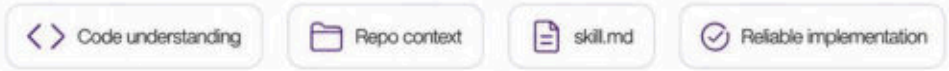
Explains output

skill.md Makes Good Work Repeatable

Use reusable instructions so Claude Code follows your project conventions every time.

- **Capture** project conventions
- **List** common commands and workflows
- **Define** quality checks and done criteria
- **Share** reusable skills across the team

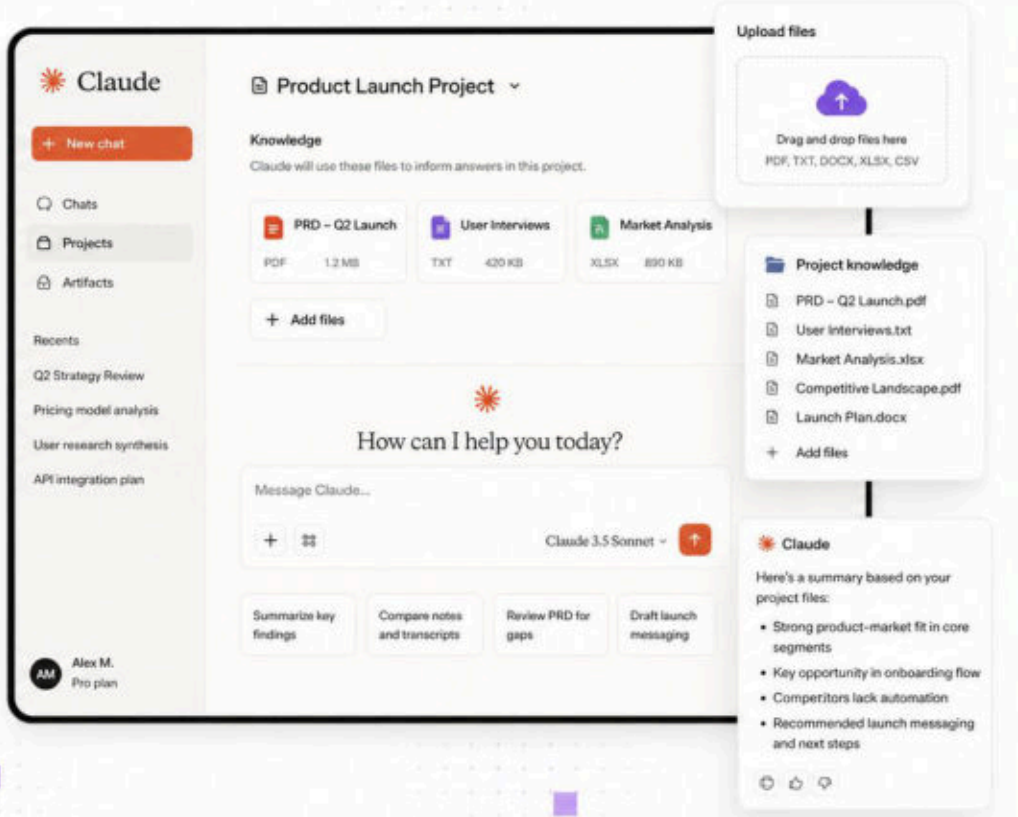




Co-work with Files and Context

Ground Claude in your project material.

- Upload notes, PRDs, and transcripts
- Keep project knowledge in one place
- Ask Claude to review, compare, and summarize
- Get answers grounded in your documents



Best Practices for Claude Code + skill.md

Use context well to get reliable code changes.

Start small —
Begin with a focused, well-scoped change.

Show key files —
Share only the files that matter.

Set constraints —
Define boundaries, style, and technology.

Use skill.md —
Keep domain knowledge and patterns in one place.

Review diffs —
Check changes carefully before applying.

Test outputs —
Run tests and validate the results.

Repo context | Commands | Acceptance criteria | Tests

Live Demo — Build a Mini-App

Examples:



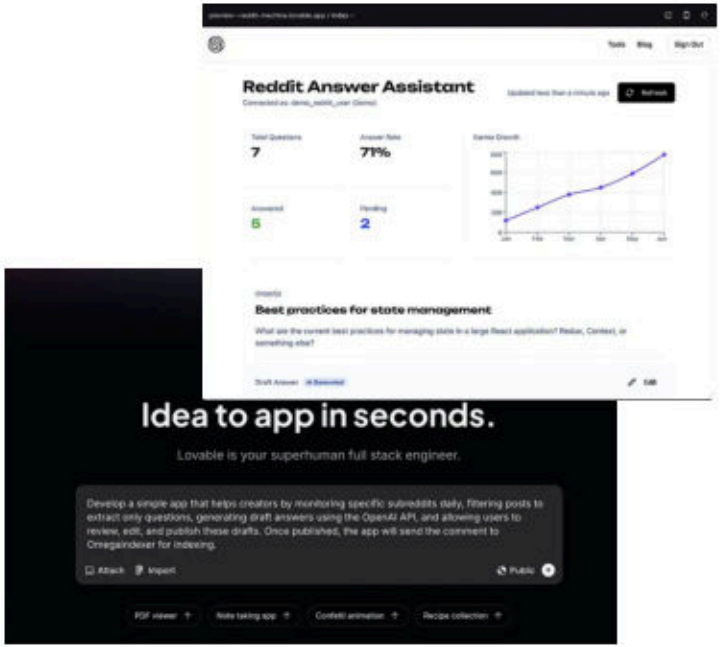
Calculator



Task Tracker



Weather Fetcher (API call)

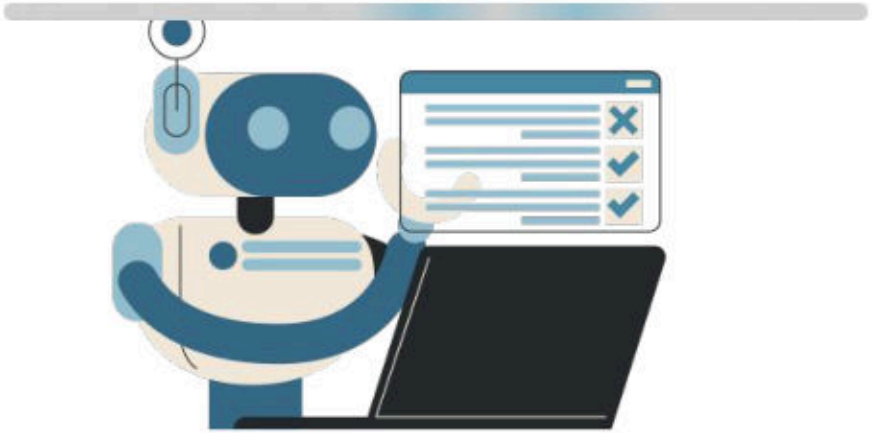


Goal:

Show idea → prompt → app within minutes

The Limits of Building with AI

- Limited backend control & logic depth
- Debugging complex systems is tricky
- Lacks versioning, testing frameworks
- Not yet production-grade



PM Use Cases

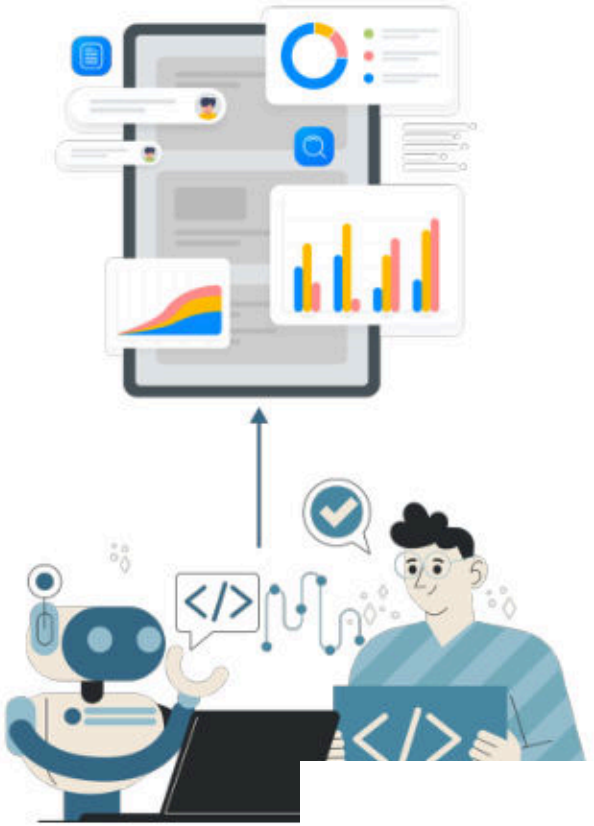
PM Use Case	What You Can Build
Internal tool	Meeting note organizer
Dashboard	Metrics from Google Sheet
Prototype	Onboarding flow
Automation	Slack alerts, email reports

Session 1 Takeaways

- Vibe coding helps PMs move from *idea* → *interactive prototype* faster.
- Prompts are your new design language.
- You can ship functional tools without waiting on teams.

FROM PROMPTS → PRODUCTS

“Debugging, refining, and giving your app a memory.



Day 2 :- Agenda

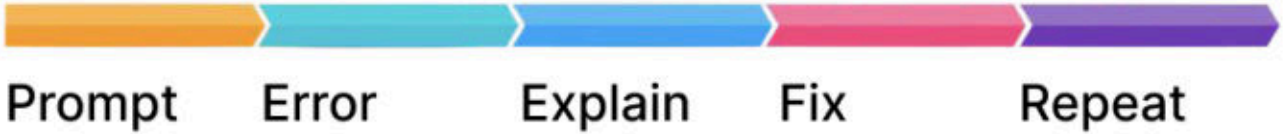
- Debugging & Refining
- Adding Data & Backend (Supabase Intro)
- Building PM-specific tools
- Hands-on Project + Interview Prep

Debugging Mindset

Message: AI mistakes = communication gaps.

Tip:

- Don’t rewrite everything — fix one issue at a time.
- Ask AI to explain its code.
- Iterate like a conversation.

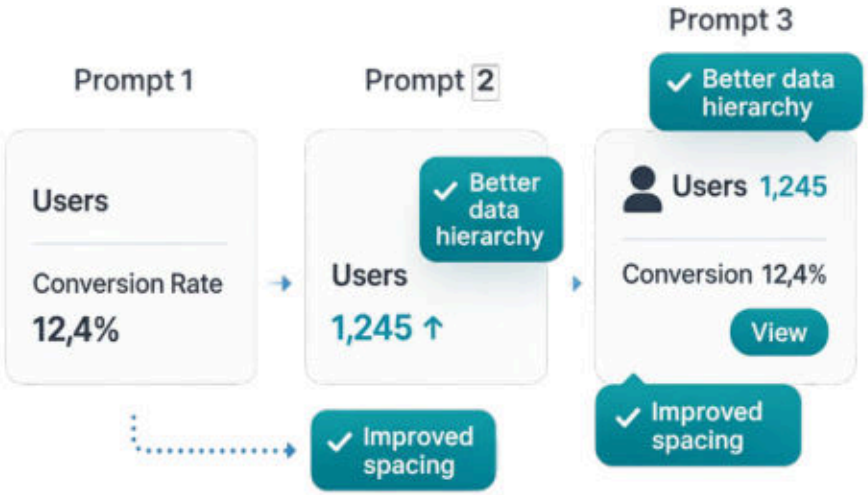


Common Issues & Fixes

Problem	Cause	Fix
Code error	Missing context	Add step-by-step logic
Wrong format	Vague instruction	Specify exact format
Unexpected output	Overloaded prompt	Break it down into sub-tasks
Performance lag	Too many dependencies	Simplify logic flow

Best Practices for Reliable AI Apps

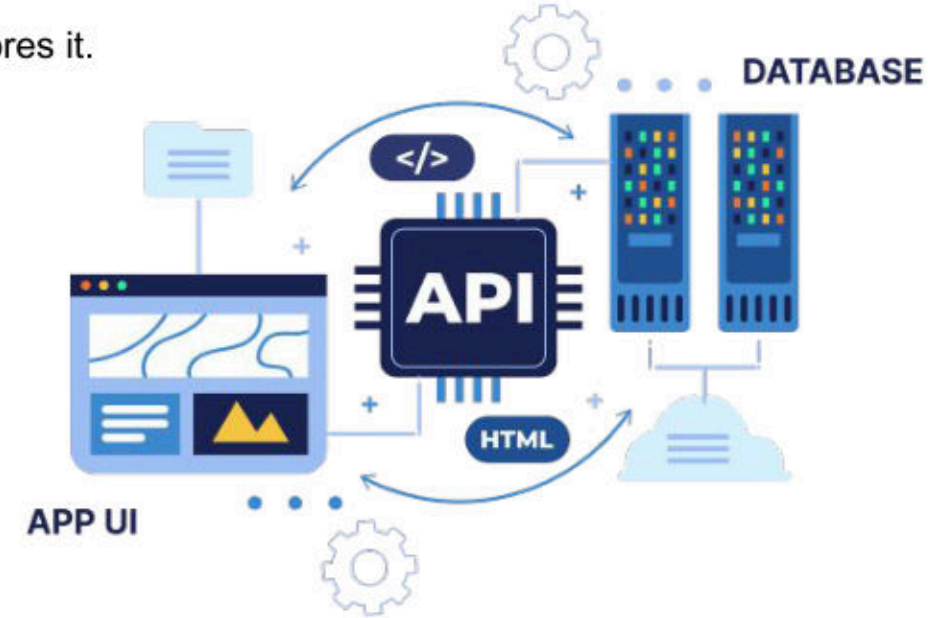
- Save prompt versions (document iterations).
- Keep each prompt scoped to one clear outcome.
- Add acceptance criteria (“must show total count correctly”).
- Use test inputs after each major change.



Supabase & Backends — Giving Your App a Memory

Core Idea: Frontend shows data, backend stores it.

- Supabase provides instant database & auth setup.
- Use it to store notes, feedback, or tasks.
- No SQL knowledge needed — define data like a spreadsheet.



API Connectors in Vibe Coding

Core Idea: Your app talks to other apps via APIs.

Examples:

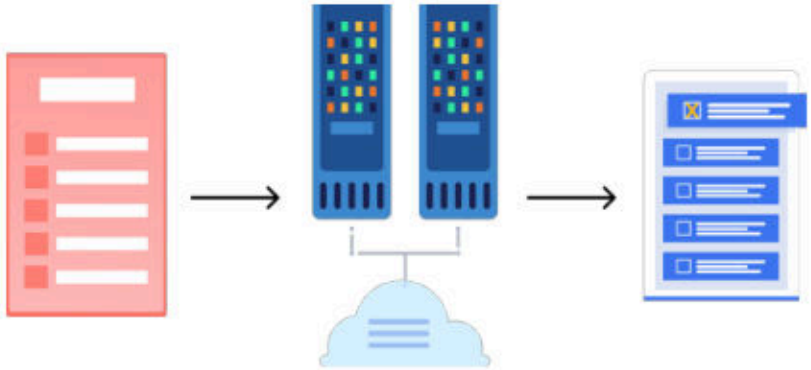
- Fetch weather → OpenWeather API
- Save tasks → Supabase
- Update Sheets → Google Sheets API



Hands-On Mini Extension: Connect Data

Task:

1. Create Supabase table (Tasks / Notes).
2. Link via Lovable's "Connect Data."
3. Add & read entries from app.



Takeaway: "You just built a full-stack app without writing code."

Build Your Own Tool (Project)

Choose one idea:

- Feature Prioritization Dashboard
- Team Task Notifier (Slack + Supabase)
- Product Feedback Summarizer (Sheets + Lovable)

Deliverable: Working demo + prompt documentation.

Showcasing Your AI-Building Skills

What Interviewers Look For:

- Clear problem framing → prompt quality
- Debugging logic → clarity of iteration
- Product thinking → impact and usability



Wrap-Up + Next Steps

- Built and debugged AI-generated apps
- Connected them to live data via Supabase
- Learned APIs as building blocks
- Built portfolio-ready PM tools

Bridge: “Next, we’ll scale these into AI-first systems using multi-agent flows and workflow automation.”



Thank you!